ROBOLAB / LOCAL RESEARCH STUDIO

From code to experiments

A complete guide to the robot lab, detailed people and editable scenes

Easy explanations • worked calculations • source walkthroughs • operating procedures



Actual bundled character geometry and textures, rendered on CPU. These are static previews, not a browser performance benchmark.

Document control	Value
Software	RoboLab 7.0.0
Prepared	26 October 2026
Audience	Owner, student, developer and dataset researcher
Scope	Local app, rover, sensing, datasets, terrain, assets and scene editing

This handbook explains what the shipped code does and why. It separates software verification, model assumptions and hardware validation.

Contents

1. Start here: the whole system 5

2. Installation and daily workflow 6

Run on Windows . . . 6
Your first recording . . . 6
Source editing versus running . . . 6
If an old version remains visible . . . 6

3. Code walkthrough: state to dataset 7

The central idea: state, update, observation . . . 7
1. How the laptop starts the website . . . 7
2. Reading JavaScript without getting lost . . . 7
3. One frame through the program . . . 8
4. Units and coordinate conversions . . . 8
5. Kinematics before dynamics . . . 8
6. Motor, tyre and body are different states . . . 9
7. Why four contact patches matter . . . 9
8. The route is a teacher, not a rail . . . 10
9. Potholes: surface versus support . . . 10
10. What the sensors mean . . . 11
11. A worked camera example . . . 11
12. What goes into a dataset ZIP . . . 11
13. Calibration numbers you can reproduce . . . 12
14. Comparing a real run with a simulated run . . . 12
15. A repeatable research session . . . 12
16. Extending the project safely . . . 12
17. What remains approximate . . . 13

4. Robot geometry and physics foundations 14

Geometry and provenance . . . 14
Drive model . . . 14
Calibration workflow . . . 15
Data and tools . . . 16
Limits . . . 16
References . . . 16

5. Research tools, sensing and data quality 17

Twelve requested areas . . . 17

Measurement workflow . . . 18
Timing, noise and sensor validity . . . 18
Four-contact reduced-order physics . . . 19
Camera and labels . . . 19
CAD import . . . 19
Reproducibility and replay . . . 20
Exports and training . . . 20
Practical laptop use . . . 21

6. Detailed geometry, materials and prop physics 22

First run . . . 22
Shapes and materials . . . 22
Graphics and camera capture . . . 23
Add or import an object . . . 23
Physics scope . . . 23
Build and asset tools . . . 24

7. Terrain, potholes and speed trials 25

Start . . . 25
Visible panels . . . 25
Potholes . . . 25
Speed bumps . . . 26
Route speed . . . 26
Trial reports and recordings . . . 26
More useful contacts for imported props . . . 26
What still needs your real-world data . . . 27

8. The furniture and asset library 28

New models . . . 28
Choose and place . . . 28
Mirror . . . 29
Source and offline operation . . . 29

9. Detailed people and scene arrangement 30

Run the updated download . . . 30
What changed . . . 30
Add and place a person . . . 30
Rearrange original furniture . . . 31
Saving and restoring a layout . . . 31
How the human code works . . . 31
How editing works . . . 32
Source map . . . 33

Performance and privacy . . . 33

10. Worked learning exercises 34

Exercise A: calculate expected wheel speed . . . 34
Exercise B: find a synchronization error . . . 34
Exercise C: identify leakage . . . 34
Exercise D: move a model properly . . . 34
Exercise E: repeat a pothole experiment . . . 34
Exercise F: explain a human animation limitation . . . 34
A practical study schedule . . . 34

Appendix A. Actual source excerpts 35

A1. Pose and impulse helpers . . . 35
A2. Fixed-step advance . . . 35
A3. Validated transforms . . . 35
A4. Independent human instances . . . 36
A5. Camera projection contract . . . 36
A6. CSV escaping . . . 36

Appendix B. File and concept atlas 37

Glossary . . . 39

Appendix C. Verification and provenance 40

Completed checks . . . 40
Verification limits . . . 40
Local acceptance check . . . 40
Attachment provenance . . . 41
Human asset provenance . . . 41
Source references . . . 41

1. Start here: the whole system

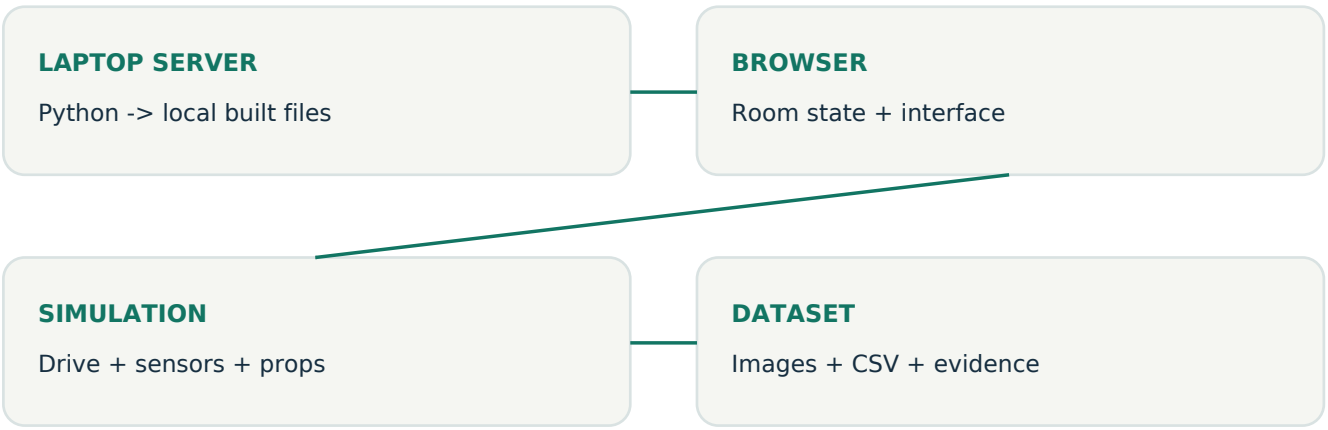
This handbook documents the uploaded RoboLab V6 project and the V7 people/scene update made from it. It concerns the robot simulator. The separate SAM boundary/Fourier tool and interactive study-book application are outside this codebase.

The project runs on your laptop. The local server provides built files; the browser draws the scene, updates the robot and collects recordings. V7 keeps the established dark studio while replacing primitive people with detailed rigs and adding direct scene arrangement.

Read Chapter 2 to run the app. Chapter 3 explains the code step by step. Chapters 4-8 give the physics, research, visual, terrain and asset details. Chapter 9 explains new people and layout controls. The appendices contain actual source excerpts, a file atlas, verification limits and references.

Source facts come from the attached project and the updated implementation. Worked calculations illustrate those equations. Automated tests check mathematical/software properties. Hardware validation would require actual measurements and recordings, which were not supplied for this update. A realistic-looking mesh or a passing unit test does not substitute for those measurements.

The earlier V2-V6 notes preserve the history of changes. Where a later feature supersedes an earlier limitation, the handbook states the current behaviour. In particular, current sensing includes the V3 timing/error pipeline, current furniture includes all 28 V6 models, and current people use the V7 detailed character system.



The server sends files; simulation and capture run in the browser.

Figure 1. The file server, browser simulation and exported evidence have separate jobs. No remote AI service is required for this robot simulator.

2. Installation and daily workflow

Run on Windows

1. Stop the old server with Ctrl+C before replacing files. Keep any old recordings separately.

2. Extract **all** of this ZIP into a fresh folder. Open the folder containing `start-windows.bat` in VS Code.

3. In PowerShell run:

```
.\start-windows.bat
```

Open http://127.0.0.1:8774/ and keep the terminal open. Default local account: **admin / robot123**. Python 3 is sufficient for the included build; no npm installation is required to run it.

- Simulator: http://127.0.0.1:8774/

- Calibration, comparison, ZIP inspection, replay and split tools: http://127.0.0.1:8774/research.html

- Numerical verification: http://127.0.0.1:8774/verification.html

- Model and material showroom: http://127.0.0.1:8774/showroom.html

The server binds to **127.0.0.1 only**. No cloud account or public deployment is needed. Imported CSVs, images and CAD are processed locally. Optional narration uses a browser voice only when it is marked as local.

Your first recording

Choose a room, inspect the robot and confirm that camera and ToF measurements change as expected. Start with a short 640x480 JPG recording. Choose telemetry rate and image interval explicitly, stop, export and inspect the ZIP before collecting a large run. Chapters 3 and 5 explain the files and timing.

Source editing versus running

Use npm ci and npm run dev for source development. Use npm run build before distributing or using the Python launcher after an edit. The complete source and ready-to-run dist are both in the updated ZIP.

If an old version remains visible

Stop the old terminal, extract to a fresh folder, run the launcher from that folder and check the V7 header. Hard-refresh the browser. If a port is busy, run .\start-windows.bat 8775. The printed local URL is the one served by that terminal.

3. Code walkthrough: state to dataset

The central idea: state, update, observation

Think of the simulator as a small world described by numbers. The robot has a position, heading, body velocity, wheel rotation and battery energy. A person has a scene transform, route and animation phase. A camera has a position, orientation, field of view and image size. JavaScript updates those numbers; Three.js turns scene geometry into pixels. An attractive picture is an observation of that state, not proof that all of its physics is exact.

The same distinction helps when reading the code. A command says what a motor should attempt. A dynamic state says what the approximate model actually did. A sensor measurement adds sampling time, noise and validity. Ground truth is the internal simulated state. Keep these four things separate when training a model or checking accuracy.

1. How the laptop starts the website

The BAT file changes into its own directory and tries the Windows Python launcher, then Python. server.py serves only the built dist directory. It binds to 127.0.0.1 so another computer cannot normally open it over your network. That address is your own machine; it is not a deployed cloud service. Closing the server terminal stops serving new page requests.

The browser loads index.html, its CSS and compiled JavaScript. boot.js creates or checks the local UI account, chooses an environment, loads the two detailed character templates, then dynamically imports main.js, canteen.js or track.js. Dynamic import means the selected room's code can be loaded when needed. A missing character produces a visible warning and the old fallback person; it does not silently download an online replacement.

Source JavaScript lives in src. Vite builds browser bundles into dist. Therefore a source edit in VS Code does not change the prebuilt app until you rebuild. During development, npm run dev serves current modules with automatic refresh. This distinction explains why copying an older source file into a folder can still show a newer interface: the BAT is serving whatever is in dist.

2. Reading JavaScript without getting lost

An import line connects modules. An exported function is a named entry point another module can use. A class groups related state and operations, such as DriveDynamics or SensorSuite. A callback is a function passed to another component to run later. The physics callback is run once per fixed step; an image-encoding callback can finish after its capture time.

An object literal holds named fields: a parameter object can contain mass, radius and friction. An array holds ordered values: four contact loads always have a specific wheel order. A Map associates an ID with a record. Set membership avoids registering the same mesh twice. A Promise represents an operation that will finish later. Await pauses that asynchronous function until the result is available; it does not turn the entire browser into a physics thread.

Start by reading the module's exported functions and their parameters. Follow the function called by the room, then inspect helpers it calls. Do not try to understand every decorative mesh before understanding the frame loop and data flow. Many older files use dense lines, so editor formatting or code folding is useful. Keep a clean source copy before making changes.

3. One frame through the program

The browser requests the next animation frame. The room admits a bounded amount of elapsed time. Robot dynamics uses a 1/120-second accumulator: admitted time is spent in small equal substeps. Fixed substeps make the equations less dependent on display frame rate. Excess wall-clock time can be dropped rather than making the browser perform an arbitrarily long catch-up loop.

At each drive substep, the current commands, surface and attitude determine motor torque, wheel acceleration, tyre forces, body acceleration and pose. The room resolves its simplified blocking/contact response and updates terrain support. Added props can be stepped through Rapier. The resulting state is available to research recording. The rendered camera and ToF acquisition are still triggered from the room loop; requesting 120 physics steps per second does not give 120 independent rendered sensor acquisitions per second.

Then the browser draws the orbit scene and the robot camera. At a recording deadline, it captures the actual available snapshot. Encoding the resulting pixels as PNG or JPG is asynchronous. The recorder preserves the capture snapshot and waits for pending encodes before final packaging. A timestamp should describe when a frame was observed, not merely when disk writing finished.

4. Units and coordinate conversions

Research world coordinates use x east, y north and z up. Three.js scene coordinates use x, height and z south. Consequently a scene position at x=2, z=-3 corresponds to research x=2, y=3. The sign change is intentional. A camera's optical coordinates use x right, y down and z forward, which are different again.

The body frame uses x forward, y left, z up. A positive research yaw rotates the forward direction toward north. Internally trigonometric functions use radians; many exported human-readable angle columns use degrees. Multiplying radians by 180/pi gives degrees. Never combine a degree value with a radian value because their numbers look plausible.

Distance is metres, time seconds, mass kilograms, torque newton-metres, force newtons, angular velocity radians per second unless a field says otherwise, current amperes and energy watt-hours. A 135 mm wheel diameter is 0.135 m, so the radius is 0.0675 m. Confusing diameter with radius doubles peripheral speed and can contaminate every odometry result.

5. Kinematics before dynamics

For a differential robot, the two side speeds suggest forward speed v=(vR+vL)/2 and yaw rate w=(vR-vL)/track. These are kinematic relationships: they describe geometry if rolling assumptions hold. A force model adds the question of whether the motor can actually produce those speeds on the selected surface.

For constant v and w over a short interval h, the robot follows an arc. integratePose uses a midpoint direction and a sinc correction to integrate that arc smoothly, including the nearly straight case. The sinc function here is sin(a)/a, approaching 1 as a approaches zero. The small-angle expression avoids division problems near a straight line.

At the default radius and 60 RPM, peripheral no-load speed is 2*pi*0.0675*60/60 = about 0.424 m/s. That is a theoretical wheel-edge speed, not a guaranteed robot cruise speed. Motor torque, voltage sag, rolling resistance, slip and the speed controller can all reduce body speed.

6. Motor, tyre and body are different states

DriveDynamics stores two motor-side angular speeds, representing the two wheels and motors on each side. Motor torque decreases with increasing shaft speed and changes with applied duty and terminal voltage. Wheel inertia means a command changes angular acceleration rather than teleporting wheel speed to its target.

A tyre's peripheral speed is radius multiplied by angular speed. Subtract the local body speed to find contact mismatch. The model asks for a force proportional to this mismatch, then limits it according to available friction and normal load. With too little grip, the wheels can spin while the chassis travels much less. This is why encoder odometry is integrated from wheels independently of the ground pose.

Slip ratio divides the mismatch by the largest of a minimum speed and the absolute wheel/body speeds. The minimum denominator is 0.05 m/s, keeping near-zero divisions bounded. Near standstill, this diagnostic is especially sensitive and should not be treated as a directly measured material constant.

Battery energy integrates voltage times estimated current times seconds, divided by 3600 to obtain watt-hours. A 12 V, 2 A load for one minute uses approximately 0.4 Wh. The model's current and voltage are approximate estimates; this is not a detailed battery chemistry, thermal or regenerative braking simulation.

7. Why four contact patches matter

Under acceleration, load tends to transfer rearward; when turning, it tends to transfer toward the outside. The model estimates four wheel-patch normal loads from supported weight, centre-of-mass height, forward acceleration and lateral acceleration. A negative estimated load sets a support-risk flag. Remaining positive loads are normalized to the supported total.

The combined longitudinal and lateral force at a patch obeys sqrt(Fx*Fx+Fy*Fy) <= mu*N. A tyre already using much of its available grip to turn cannot simultaneously supply unlimited acceleration. This is the practical purpose of the friction-circle approximation. It is more informative than assigning a speed directly to the route.

For a level 3.99822528 kg robot, supported weight is approximately 39.21 N. At mu=0.75, the total ideal horizontal friction budget is around 29.41 N if all patches contribute. Motor torque may impose a much lower limit. These numbers are model calculations, not measured tractive force.

The system does not simulate dynamic rollover or articulated suspension. A risk flag means the reduced load estimate has crossed a condition, not that a real robot has fallen over. Body pitch and roll on terrain are quasistatic approximations.

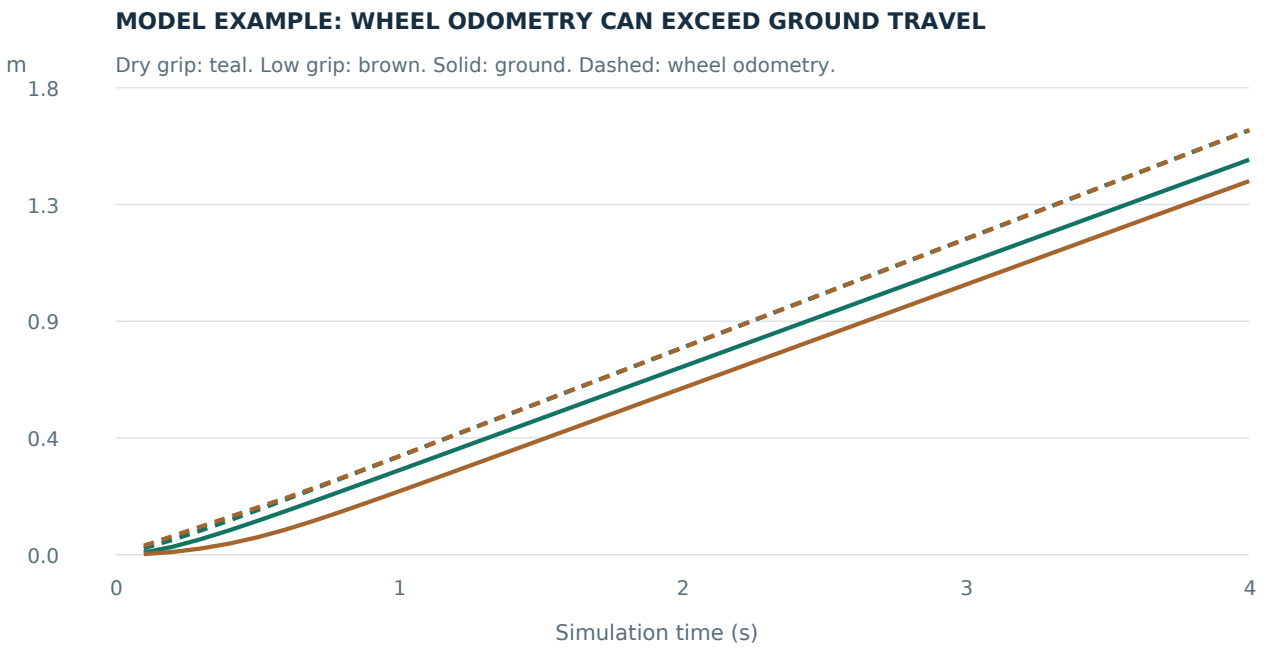


Figure 2. Recomputed from the shipped drive model for four seconds at equal full duties: mu=0.75 versus mu=0.08, all other defaults unchanged. Simulated illustration, not hardware data.

8. The route is a teacher, not a rail

research.js converts a path target into commands. It looks around the robot's nearest route progress, chooses a point ahead, computes heading error and requests a forward/yaw speed. A large heading error can request a pivot. Wheel-speed feedback converts the requested motion to bounded left/right duties.

The route curve is therefore a target. It does not overwrite the robot position on every step. Low grip, blocked motion or insufficient torque can produce tracking error or a stall. Moving the robot deliberately to a task start is a separate setup action. This distinction is essential if you later train a learned controller: the current route follower is a geometric teacher, not a trained neural network.

SpeedReference adds target-speed profiles and acceleration/braking limits. Adaptive factors reduce route speed in pothole, bump, gravel, slippery or narrow zones. Manual duty driving is a different control mode and does not become equivalent to the route speed reference simply because both move the rover.

9. Potholes: surface versus support

track-geometry.js defines a smooth, irregular elliptical depression. The rendering mesh follows the analytic height profile, and the relevant floor mesh has a real opening rather than a dark circular sticker over flat ground. Adjustable radius, depth, aspect and edge shape change the test geometry.

A wheel is not a mathematical point. tyreSupportHeight samples the upper envelope of a rolling circle at several longitudinal and lateral positions. A wheel can bridge a narrow gap instead of dropping to the lowest point directly under its centre. The support envelope captures part of that geometric behaviour without modelling deformable rubber or suspension.

Rapier props interact with a sampled heightfield of the track. The drive rover uses its own terrain-support calculation. A successful simulated crossing does not establish a safe real-world speed for a physical pothole. Compare repeated trial metrics at matched settings before drawing even simulator-only conclusions.

10. What the sensors mean

The geometric ToF query casts nine rays over a horizontal fan and chooses the nearest hit. SensorSuite adds rate, latency, jitter, offset, scale error, noise, dropout and approximate material-related loss. It keeps acquisition time separate from delivery time and prevents an old, late packet from replacing a newer reading.

An unavailable reading is null or an empty numeric export with an explicit status. Zero means zero distance; it does not mean missing. Keep a validity mask for learning. The separate capped geometric distance is useful for debugging but cannot tell you how far beyond the sensor's maximum range an unseen object really lies.

The IMU adds bias/noise to simplified accelerometer and gyro quantities. A level stationary accelerometer includes approximately +g in its vertical component. It is not zero merely because the robot is not moving. Measured encoder ticks can lose a configured fraction; the exported measured odometry is integrated from those tick changes.

11. A worked camera example

For a vertical field of view F and image height H, fy=(H/2)/tan(F/2). The implementation assumes square pixels so fx=fy. At 640x480 and 70 degrees, fx and fy are approximately 342.756 pixels. A centred principal point is cx=320, cy=240. K is a matrix containing these focal lengths and principal-point coordinates.

The distortion model uses radial and tangential coefficients. Changing FOV changes projection; changing exposure changes brightness. A normal texture changes surface shading, not collision geometry. Previous-frame blending approximates blur between samples; it does not simulate a real shutter integrating continuously over time or rolling-shutter row timing.

An annotation box comes from projected 3D bounds and heuristic visibility. It is not a SAM segmentation mask or an exact outline. SAM boundary tracing and Fourier epicycles belonged to the separate earlier application; they are not silently included in this robot simulator's labels.

12. What goes into a dataset ZIP

data.csv preserves the legacy combined table. inputs.csv selects observations and identifiers; commands.csv holds applied control fields; ground_truth.csv selects internal truth and diagnostic fields. metadata.json records software version, source fingerprint, parameters, timing conventions and limitations. annotations.jsonl and event files carry generated labels. quality.json records audit results.

Images are actual encoded captures. The exporter converts binary Blobs to bytes, waits for pending work, validates image references and constructs a ZIP. A SHA-256 manifest records bytes and hashes for the other files. Hashes help detect changes or incomplete transfer; they do not prove that a simulated scene matches real hardware.

Splits group frames by environment or run rather than shuffling adjacent frames indiscriminately. Adjacent frames from one route are strongly correlated. Putting the same scene and nearly identical viewpoints into both train and test can produce deceptively good results. A 70/15/15 grouping rule is a probability assignment, so a tiny dataset can have an empty split.

13. Calibration numbers you can reproduce

If a straight two-metre travel produces a mean 5700 ticks at 1200 ticks per revolution, the effective rolling radius is 2*1200/(2*pi*5700)=0.067013 m, about 67.01 mm. This assumes negligible slip, accurate ticks-per-revolution and an accurate distance measurement.

For a full 360-degree turn with left=-2230 and right=2230 ticks, radius 0.0675 m and 1200 ticks per revolution, effective track is about 0.250875 m. A skid-steer effective track can depend on floor and motion, so agreement with CAD alone does not fully identify the real turning behaviour.

If 12000 ticks arrive in ten seconds at 1200 ticks per revolution, shaft speed is 60 RPM. If a robot coasts from 0.4 m/s to rest over 0.32 m on a level floor, the simple effective Crr estimate is v^2/(2*g*d)=0.02549. Motor drag and other losses are lumped into this estimate. Use several repeated measurements and retain raw evidence.

14. Comparing a real run with a simulated run

First match units and choose acquisition-time columns. Convert to relative time so different wall-clock start dates do not dominate. An explicit offset accounts for a known synchronization difference. The comparison interpolates only within an allowed gap and does not extrapolate beyond available data. Yaw must be unwrapped before interpolation so 359 to 1 degrees is a small rotation rather than a large jump.

Bias is the mean signed error, MAE is the mean absolute error, RMSE is the square root of mean squared error, and max error shows the worst aligned difference. RMSE is more sensitive to large outliers. Report the signal, unit, alignment rule, number of samples and allowed gaps alongside the score. Removing initial pose offsets changes the question from absolute localization to relative trajectory agreement; disclose that choice.

15. A repeatable research session

Begin with a named environment, saved layout, route and calibration. Use a fixed seed and record the complete randomization profile. Keep one baseline with effects disabled, then change one factor at a time: grip, payload, noise, lighting, speed or pothole depth. Do a short recording first and inspect images, timestamps, statuses and missing files.

Use 640x480 JPG while checking the pipeline. Choose telemetry and image intervals deliberately: a 4 Hz target means a 0.25-second interval. The legacy initial image interval is two seconds, so do not assume every telemetry row automatically contains a new image. Keep the browser tab active and inspect missed/late counters before increasing resolution or scene complexity.

Export the dataset, scene, calibration and trial report. Keep your hardware logs separately with the same experimental identifier. Inspect flagged images manually. Hold out entire environments and real-world runs for evaluation. Numerical consistency checks support software correctness; only matched hardware evidence supports claims about physical accuracy.

16. Extending the project safely

To add a sensor error, put the model in SensorSuite and export its parameters, status and timestamps. Test missing/late observations rather than only the happy path. To add a furniture asset, keep its ID stable, set reasonable nominal dimensions, retain licensing and make its source geometry available. A material texture should not alter collider dimensions accidentally.

To change drive equations, work in dynamics.js first and run its numerical tests. Verify straight symmetry, turning sign, fixed-step consistency, finite state, friction/energy limits and adverse settings.

Then inspect the room integration. To change an export field, follow it through column definitions, row construction, split selectors, metadata and tests. A new field that is never filled or is placed in the wrong training partition can be worse than no field.

To add a person, preserve the synchronous human API and clone skeletons independently. To change layouts, use the scene editor's commit path so collision state and persistence follow the visual transform. After any source change, rebuild dist before packaging a launcher-based download.

17. What remains approximate

The reduced rover does not have articulated suspension, deformable tyres, realistic battery chemistry, full optical ToF behaviour or calibrated human biomechanics. Rapier governs added props; the rover's motor/tyre model, older room objects, walkers and gates are not all unified into one six-degree-of-freedom solver. Visual realism and physical fidelity are separate properties.

A seed is not a complete experiment. Retain model files, settings, scene objects, source version and initial states. Replay includes corrections from recorded contact snapshots; that is useful for inspection but should not be described as an entirely independent recreation of every environment interaction. The provided tests and CPU previews do not certify GPU performance on your Lenovo, nor do they replace real recordings.

Use these limitations constructively: identify which observation matters for your task, measure that observation on hardware, and improve the smallest model component that explains the error. Better evidence usually improves a research simulator more than adding another decorative effect.

4. Robot geometry and physics foundations

Historical V2 reference. Current behaviour and verification are documented in RESEARCH_V3.md and VALIDATION_V3.md.

This update preserves the original dark simulator and its three environments. It replaces the generic robot with a parametric rover using the wheel-joint spacing from `my_bot.urdf`. It adds a source CAD inspector, shared force-based driving, calibration, and numerical verification. The full source and a prebuilt local app are included.

Geometry and provenance

Quantity	Starting value	Basis
Total robot mass	3.99822528 kg	Sum of inertials in my_bot.urdf; not a physical weighing
Wheel centre track	0.251018 m	Left/right wheel joint origins in my_bot.urdf
Wheelbase	0.151022 m	Front/rear wheel joint origins in my_bot.urdf
Wheel radius	0.0675 m	Starting assumption of 135 mm diameter; editable
Output no-load speed	60 RPM	Starting assumption; editable
Torque / current / friction / battery	See Physics tab	Engineering estimates requiring calibration

Six referenced meshes are absent from the uploaded project: `wheel.stl`, `johnson_motor.stl`, `aluminium_sheet.stl`, `holder1.stl`, `holder.stl`, `motor_bracket.stl`. Their shapes cannot be recovered from the URDF joint layout alone. The driven robot uses labelled parametric replacement visuals. No missing mesh is claimed to be reconstructed exactly.

The separate `robot.urdf` assembly has its complete referenced STL set. The inspector uses the actual visual/collision geometry, URDF origins, fixed-axis RPY rotations and hierarchy, in metres. It can show collisions, joint axes and the source centre of mass. `my_bot.urdf` can also be inspected: missing geometry is marked by amber wire boxes. Both original URDF files and every original STL remain unchanged. See SOURCE_AUDIT.json.

Drive model

The drive state advances at a fixed 1/120 s. A frame accumulator gives identical results at ordinary 24/30/60/144 Hz frame rates for the same admitted time and commands. Frames over 50 ms are capped by the room loop to keep contact work bounded; discarded wall time is reported. Simulation time is explicit and can run slower than real time on a slow or backgrounded tab.

There are two wheel-side angular states (two motors on each side), body forward velocity and yaw rate. The approximate equations are:

- Motor torque decreases linearly with shaft speed, scaled by motor duty and available battery voltage. Zero duty coasts unless electrical braking is selected.

- Longitudinal tyre contact force is proportional to peripheral-wheel / ground-speed mismatch. The current model distributes it across four contact patches and applies a combined longitudinal/lateral friction limit at each patch; Chapter 5 gives the extension.

- Wheel angular acceleration follows `(motor torque - contact force × radius - viscous drag) / side inertia`.

- Body acceleration follows net tyre force, rolling resistance and downhill gravity divided by mass.

- Yaw acceleration follows the difference between left/right traction moments minus skid damping divided by yaw inertia.

- The ground pose is integrated from body motion. Encoder odometry is independently integrated from wheel rotation. Wheelspin therefore creates real simulated odometry error rather than copying ground truth into encoder values.

- Battery energy integrates estimated terminal voltage × current × time. Voltage sag depends on empirical internal resistance. These are estimates, not an electrical, regenerative or thermal motor model.

Manual keys apply open-loop motor duty. Route driving uses lookahead steering and wheel-speed feedback. The curve is a target, not a pose constraint, so robots can drift, stall or fail to follow a path under adverse conditions. Turn-in-place is subject to the same torque and friction model. Original task placement still intentionally relocates the robot at the beginning of a task.

Wall sweeps and object contacts retain the room's simplified collision approach. The planar collision envelope now follows rover dimensions. Object normal impacts share the tested restitution/momentum function. Falling uses time-of-impact integration against the supporting surface. Terrain attitude remains quasistatic and the gate retains the earlier approximate hinged-plate model.

Calibration workflow

1. Measure wheel diameter under load, wheel-centre track, wheelbase, total mass with payload, and output RPM at a recorded supply voltage.

2. Enter these in Physics. Stop recording before applying a new calibration.

3. Record a straight acceleration test, coast-down test, turn-in-place test and low-grip wheelspin test on hardware.

4. Fit torque/current, wheel inertia, rolling resistance and yaw damping to those recordings. A visually smooth path is not sufficient calibration.

5. Export calibration JSON and retain it with the real recordings. All dataset metadata includes parameters, provenance, selected surface and model limitations.

Calibration is validated and saved in this browser. Applying it resets motion and wheel odometry; simulation clock and battery energy remain continuous. Original room mass controls are also respected. Defaults restore the starting assumptions.

Data and tools

The original 58 CSV columns retain their positions; ten new physics columns bring the total to 68. Wheel-side velocity and RPM now come from simulated shaft rotation, PWM from the applied motor commands, and odometry from independent wheel integration. New fields contain motor angular speed, slip ratio, traction, normal force, fixed step, model identifier and wheel-odometry error. Slip ratio is signed and normalised by at least 0.05 m/s; it is an engineering diagnostic near standstill.

Camera and ToF transforms follow body attitude and the updated rover dimensions. The four ToF mounts come from the supplied URDF. The geometric ToF query uses nine rays over a 25° horizontal field. The current V3 pipeline additionally models timing, bias, scale, noise, dropout and approximate material-related loss. It remains an approximate sensor model; Chapter 5 describes its current meaning.

The Physics tab offers live diagnostics, calibration import/export, a conservative collision-envelope overlay, wheel-odometry marker and JSON report. `verification.html` runs without WebGL or login and contains eight numerical checks, a dimensioned top view, a wheelspin plot, source STL/URDF audit and report export.

Limits

This is a reduced-order research prototype, not a full 6-DOF rigid-body, suspension, deformable tyre, optical sensor or thermal simulator. It does not physically validate your robot. It cannot reproduce absent CAD geometry. Existing generated scene textures, people, curtains and other decorative animations remain synthetic. A real dataset comparison is required before making accuracy claims.

Local operation: Python serves the prebuilt app on 127.0.0.1:8774. Node/Vite is only needed for source development and rebuilding. All runtime libraries are bundled; remote Google font requests and the optional legacy cloud-download route were removed. Files export through browser downloads. The original local-storage login remains a UI convenience; the local server itself has no account-based access control and binds only to loopback.

References

- ROS URDF physical/collision properties: https://docs.ros.org/en/rolling/Tutorials/Intermediate/URDF/Adding-Physical-and-Collision-Properties-to-a-URDF-Model.html

- Gazebo physics parameters and solver modelling: https://classic.gazebosim.org/tutorials?tut=physics_params

- Source geometry and inertials: robot.urdf, my_bot.urdf and assets/ in this project.

ZIP export packages a snapshot of the selected records and converts image Blobs to binary buffers explicitly. Progress and errors are visible. When ready, click a ZIP link in the left dock to start the browser download. Separate batch links are retained until the tab closes. CSV fields escape commas, quotes and line breaks.

5. Research tools, sensing and data quality

Twelve requested areas

Area	Implemented tool	What needs real evidence
Robot calibration	Radius, effective track, RPM and coast-test calculators; evidence JSON; editable mass/COM/motor/inertia parameters	Actual distances, encoder counts, timing, weight and COM measurements
Real vs simulated runs	CSV column/unit mapping, relative-time alignment, offset, bounded interpolation, plots, RMSE/MAE/bias/max	A real run made under matched commands and conditions
Synchronization	Run/frame IDs, simulation capture timestamps, sensor acquisition/delivery stamps and late counters	Real collector acquisition timestamps rather than only file-save times
Sensor realism	ToF latency/jitter/bias/scale/noise/dropout; approximate dark-material loss; IMU and encoder errors	Error distributions measured at several distances/materials
Quality checks	CSV shape/time/finite checks, image references, decoding, uniform/clipping audits and SHA-256 manifest	Visual review of flagged frames and real GPU captures
Geometry	Persistent local STL overrides with unit selection, bounds and hash audit	The six actual missing meshes; measured dimensions/inertias
Contact physics	Four patch normal loads, combined traction limits, lateral motion, wall response and geometry-aware gate size	Matched traction/turn/braking/ramp experiments
Camera calibration	FOV/principal point, body mount, exported K/D matrices, distortion and image effects	Real camera calibration and mount measurements
Ground truth separation	`gt_*`, ground_truth.csv, inputs.csv, commands.csv and explicit label roles	Task-specific prevention of target leakage
Reproducibility	Seeded streams, source build ID, settings, experiment snapshots, initial scene archive and input replay	Hardware replay and independent environment/contact state where needed

Area	Implemented tool	What needs real evidence
Environment variation	Bounded seeded mass/friction/exposure/light/material/object perturbations	Plausible ranges; randomization does not certify realism
Dataset splits	Stable environment/run group split assignment and cross-dataset manifest builder	Enough independent environments and held-out real-world tests

Measurement workflow

Use the standalone research page even if WebGL is unavailable. Import the existing calibration first. Otherwise unmeasured fields remain defaults. Enter real measurements and repeat trials.

- Radius: `r = distance * ticks_per_revolution / (2π * mean_absolute_ticks)`. Straight level motion with negligible slip is assumed.

- Effective track: `b = |right_ticks - left_ticks| * 2πr / (ticks_per_revolution * measured_angle_rad)`. This is floor- and skid-dependent, not necessarily the CAD track.

- No-load RPM: `60 * ticks / (ticks_per_revolution * duration_s)`. Measure at the stated nominal voltage.

- Coast estimate: `Crr = initial_speed² / (2g * stopping_distance)`. This lumps motor drag into an effective resistance.

- Weigh the full robot including battery and payload. Enter total mass under Physics. Torque, inertia, tyre stiffness and battery response remain estimates until independently identified.

Download the measured profile and import it under Physics. Evidence is preserved in subsequent exports. A manual parameter edit clears previous evidence from the active profile; keep the evidence file.

Timing, noise and sensor validity

Dynamics uses 1/120 s substeps. Renderer stalls are capped and dropped wall time is reported. Samples are generated from actual available physics snapshots, not interpolated or copied to pretend a requested rate was met. Sensor ray acquisition is render-triggered; sensor rate is a target, not a separate hard-real-time thread.

Configure telemetry rate independently of image interval. Images are taken on the next recorded sample at or after the image deadline, so image rate cannot exceed telemetry/rendering rate. Both the image and its row use the same simulation snapshot. Encoding/file saving can finish later without changing the acquisition timestamp.

ToF returns values only with `ok` status. `pending`, `dropout` and `out_of_range` have empty numeric measurements. Do not interpret an empty field as zero. The capped geometric result is stored separately under `gt_tof_*`. A maximum-range geometric result does not establish the true distance beyond the sensor range. Range noise uses a Gaussian baseline plus a distance-squared component, with millimetre quantization; this is a tunable approximation, not a manufacturer-validated optical model.

IMU includes noisy accelerometer components, gyro bias/noise and integrated yaw. A stationary level accelerometer measures approximately +g on z. Measured encoder ticks can miss a configured fraction;

odometry integrates the measured tick changes and is anchored to the known simulation start pose. Truth and encoder drift stay separate.

Coordinate frames: world x east/y north/z up; robot x forward/y left/z up; camera optical x right/y down/z forward. Angles in CSV are degrees unless the name says radians. ToF mount yaw configuration is radians.

Four-contact reduced-order physics

The model retains two motor-side angular states representing four motors, with explicit wheel inertia and torque-speed response. There are four contact load estimates in FL/FR/RL/RR order. Longitudinal/lateral load transfer uses the configured COM height, forward acceleration and lateral acceleration. Negative estimated support loads trigger a risk flag; remaining loads are renormalized to total support force.

At each patch, combined longitudinal/lateral forces are limited by the friction circle `sqrt(Fx² + Fy²) <= μN`. Lateral patch forces oppose body lateral motion and yaw scrub. Body planar motion includes lateral momentum and rotating-frame coupling. Walls reject blocked translation and project/restitute velocity; gate dimensions follow the current rover envelope. Fixed substeps reduce displacement per collision query.

This is NOT a full rigid-body contact solver: no articulated suspension, arbitrary 3D mesh contact manifold or dynamic rollover is implemented. Terrain pitch/roll remains quasistatic. The support-risk flag does not simulate a real overturn. Room object and gate dynamics retain explicit approximations. Do not describe numerical checks as physics proof.

Camera and labels

The live camera uses the configured mount, FOV and principal point. `K`, `D` and optical-frame conventions are exported. Saved frames can receive radial/tangential distortion, exposure gain, pixel noise and optional previous-frame blending. Distortion maps are cached. Frame blending approximates blur between sampled images; it is not an integrated shutter model. Keep it off for baseline geometric measurements.

Object boxes are projected 3D axis-aligned bounds with heuristic visibility. They are not exact masks, and distorted images make them approximate. When ToF is unavailable, range-based safety Q&A is omitted instead of inventing a zero-distance obstacle. Generated captions/actions are simulator-derived annotations, not independent human labels. The route follower/safety logic is a geometric teacher controller; it does not become a learned real-world policy by adding noise to exported measurements.

CAD import

The original source files are unchanged. Missing files are: wheel.stl, johnson_motor.stl, aluminium_sheet.stl, holder1.stl, holder.stl and motor_bracket.stl. Supply your actual exports with exact filenames, select source units, and inspect the reported dimensions. Imports are held in browser IndexedDB and included in datasets with SHA-256 hashes. Re-open the CAD inspector and select my_bot.urdf. It will use the supplied geometry through the original URDF link/joint transforms.

The driven rover keeps its parametric replacement visuals; the complete reference assembly robot.urdf is a different model. Importing an STL does not identify mass, inertia or collision accuracy. Missing files cannot be faithfully reconstructed from joint metadata alone.

Reproducibility and replay

Save an experiment before a run. It contains seed/profile, route, existing labelled-object transforms, people route state, lights/curtains and random states. A frozen lighting hour is enabled by default for repeatable capture; disable it to use the existing live clock controls. Seed alone does not override every prior browser preference, so retain the complete experiment.

The compact experiment restore does not recreate arbitrary spawned geometry or internal gate/rolling contact history. Dataset `scene.json` archives the initial complete Three.js scene separately. `replays.json` stores 120 Hz motor inputs/options and post-contact states. The replay verifier re-executes the drive model, measures pre-correction discrepancy, then restores recorded contact corrections. That is useful for inspecting commands/drift, but is not an independent replay of all room physics. The 180,000-step per-run limit is explicit through a truncated flag. Browser/GPU bitwise determinism is not promised.

Randomization is off by default. Bounds are validated; selected movable-object overlaps are rejected. Each applied perturbation is recorded. Calibration remains distinct from sampled runtime parameters. Keep floor, payload, camera and lighting ranges plausible. Repeated variants of the same room should keep the same environment ID for split grouping.

Exports and training

Stop, Export, wait for packaging, then click the green ZIP link. Folder mode writes the same files after checking referenced images. Asynchronous image writes are awaited; errors are reported. Clear waits for pending encoding and then clears recording buffers. ZIP checkpoints are cumulative to prevent dangling references across batches.

- data.csv: compatibility telemetry plus all appended research fields.

- inputs.csv: measured sensors/odometry and image references; excludes true pose, true ToF, true velocity and current command.

- ground_truth.csv: simulator truth and selected diagnostics.

- commands.csv: applied control signals; useful as targets for imitation learning, not automatically safe to include as inputs.

- metadata.json: calibration/evidence, frames, settings, run identity, assumptions and source build fingerprint.

- quality.json: consistency result and image audit. Warnings require review; a genuinely dark scene may be uniform.

- manifest.json: bytes and SHA-256 for each other file; self hash excluded. This detects corruption/changes, not authenticity.

- annotations/events: simulator-derived labels and event-image associations. Events refer to the nearest preceding recorded image, not a new simultaneous capture.

- splits/: row/frame/image indices for whole-group train/validation/test assignments.

- experiments.json/replays.json/scene.json/cad/: reproducibility evidence.

Build sequences within one run. Preserve sensor acquisition/delivery stamps and validity. Do not backfill invalid values with zero or attach a later image to an earlier row. Choose targets according to the task; separated files alone cannot prevent all leakage.

Comparison accepts ISO timestamps or numeric seconds/milliseconds, and selectable metres/centimetres/millimetres or radians/degrees. It aligns relative times, removes initial translation/yaw offsets, interpolates within a maximum gap and never extrapolates. This alignment is not full trajectory registration or a proof of physical agreement.

Group splitting uses a stable avalanche-mixed hash of seed and group, with 70/15/15 probabilities. Whole environments remain together by default. Small collections can have empty splits; collect more independent groups rather than moving nearby frames. The split builder combines metadata from multiple recordings and deduplicates run IDs. Use a held-out real dataset for final sim-to-real claims.

Practical laptop use

Start with a short 640×480/JPG recording at 4 Hz and images every 0.25 s. Inspect it before scaling up. Increase resolution only if timing/quality remains acceptable. Long high-resolution PNG recordings can consume substantial browser RAM; use folder mode or shorter recordings. Dataset browser inspection accepts ZIPs up to 250 MB and 700 MB declared uncompressed size. All parsing, hashing, CAD storage and image processing remains local. Keep exported files if clearing browser data.

6. Detailed geometry, materials and prop physics

This update preserves the dark simulator, three rooms, navigation, human control, original object controls, physics settings, sensors, recordings, calibration, CSV comparison, quality inspection, replay and group splits. It adds detailed geometry, offline surface materials, model editing and Rapier contacts for scene additions.

First run

Stop the older server with Ctrl+C. Extract the full ZIP to a fresh folder and run `.\start-windows.bat` in PowerShell. Open `http://127.0.0.1:8774/`, sign in with the local default `admin / robot123`, choose a room and open **Visuals**. The source and the ready-to-run `dist/` are included; Python 3 is enough to run them. The server is loopback only.

If the page still looks old, check the terminal's served folder, stop any other server on port 8774 and hard-refresh with Ctrl+F5. The current header says **People & Scene Studio V7**. Editing `src/` requires `npm run build` before restarting the Python launcher.

Shapes and materials

- The rover has two rounded, slotted plates; four profiled tyres with 144 instanced tread blocks; hubs, bolts and standoffs; gearboxes; an illustrative battery, board, heatsink, connectors, camera and four ToF housings; and cable runs. Default visible wheel centres/radii follow the existing rover profile. Static parts are batched by material. The dynamic rover contains 25 mesh objects and about 25,400 visible triangles.

- The V4 foundation introduced eight original models: dining chair, office chair, sofa, coffee table, sideboard, planter, service trolley and timber crate. They use 1-5 mesh objects each and fewer than 6,000 triangles each. Their illustrative dimensions and masses are editable assumptions, not a furniture measurement dataset.

- Five ambientCG material sets provide colour, OpenGL normal and roughness maps, with a metalness map where applicable. Wood, plaster, woven fabric, metal and tile are bundled at 1K. UV projection uses physical repeat distances where supported. A Poly Haven 1K studio HDR environment supplies reflection lighting.

- Source URLs, CC0 license information and hashes are in `public/visual/materials.json` and `ASSET_CREDITS.md`. Original generated GLBs and previews are under `public/visual/`. No remote asset server is used at runtime.

- V7 now replaces the earlier primitive people with textured anatomical rigs. See Chapter 9 for the implementation and its limits.

Use **Open model & material showroom** to orbit the original GLBs, inspect bounds and download individual models. Without WebGL, labelled still previews show actual mesh geometry; they do not verify GPU lighting.

Graphics and camera capture

Preset	Pixel-ratio cap	Shadow map	Normal maps
Low	1.0	1024	Off for bundled materials
Balanced	1.25	2048	On
High	1.75	4096	On

Balanced is the initial choice for the Lenovo LOQ / RTX 4060. Resolution, image frequency and the room's object count also affect load. Start a short test at 640×480 JPG and inspect the result before increasing resolution. The renderer's displayed FPS is diagnostic, not a guaranteed data capture rate. Browser background throttling can reduce throughput.

The simulator uses ACES filmic tone mapping and sRGB output for preview and offscreen camera textures. Exposure and environment reflections are adjustable. Camera capture temporarily restores opaque room walls so the orbit view's cutaway does not leak into robot observations. The editor's grid/selection outline is excluded from the robot camera. Existing sensor/path guides retain their visibility controls.

Add or import an object

1. Select an included model and click **Add model near robot**, or choose an embedded `.glb` file. Imports stay on this computer.

2. Select the object in the editor. Set X/Z position, vertical position, yaw and width/height/depth in metres. Use actual measured dimensions for research.

3. Set mass in kilograms. Keep **Original / imported** to preserve an imported material, or choose a bundled material override.

4. Click **Apply object**. For gravity/contact, enable **Dynamic Rapier prop** or click **Drop / settle**. **Push** applies a 0.5 N·s horizontal impulse.

5. Click **Export scene ZIP** to keep an independent backup. The app also saves scene additions in IndexedDB, separately for each room.

Imports accept self-contained GLB 2.0, at most 40 MiB per file, 250,000 triangles, 500 meshes and 4K textures. External URLs and required Draco/Basis/Meshopt compression are rejected, avoiding remote decoders or asset requests. Export uncompressed GLB from Blender with embedded images. The scene limit is 40 added props and 200 MB of imported GLB data. Material arrays are retained. Animated GLBs play the first clip, but collision remains a conservative object box rather than per-bone collision.

The scene ZIP contains `visual/scene.json` and embedded imported models. Import adds those objects to the matching room; it does not replace the whole original room or reset existing objects. Avoid duplicate imports. Browser persistence is origin-specific: changing localhost/127.0.0.1 or the port creates separate browser storage. Export before clearing site data.

Physics scope

Rapier 0.21.0 runs locally in WASM. Added dynamic props have gravity at 9.80665 m/s², conservative oriented box colliders, computed box inertia, damping and continuous collision detection. They settle, tilt and exchange impulses with other props, the ground and a planar rover proxy. The update is hooked to the existing 120 Hz physics steps, before data recording.

The rover retains V3's calibrated motor/tyre/contact-patch equations. It couples planar contact impulses to added props. This is not a full articulated-wheel solver. Up to 500 legacy meshes supply world-axis-aligned static proxies, refreshed periodically; thin/rotated geometry and under-table clearance can therefore be approximate. Existing walkers, rolling legacy objects, gates and quasistatic terrain attitude retain their V3 models. A more detailed mesh improves appearance and ray intersections; it does not make unmeasured physics exact.

ToF tests the registered visible mesh triangles. Added prop transforms and velocities are exported as `visual_objects` in image annotations, and visual configuration/limits are recorded in `metadata.json`. Dataset ZIPs include the scene-addition JSON and imported models. Visual metadata does not replace real camera calibration or measured geometry.

Build and asset tools

`npm ci`, `npm test`, then `npm run build` rebuilds the site. Run `py -m unittest discover -s tests -p "test_*.py"` for CAD and visual-asset checks; the asset checks require Pillow for development only. `tools/build-visual-models.mjs` regenerates the original GLBs. `tools/render-model-previews.py` optionally generates CPU previews using NumPy/Pillow/trimesh. `tools/fetch-visual-assets.py` optionally refreshes the fixed licensed source assets using their official APIs and requires Pillow. None of these build tools are needed for normal offline operation.

Keep the original source/CAD audit and its limitations. Six missing rover STL files still require your originals; V4 illustrative details do not substitute for them. See `VALIDATION_V4.md` before using the simulator for research claims.

7. Terrain, potholes and speed trials

V5 keeps the V4 models/materials and V3 research tools. It concentrates on the test-course potholes, speed bumps, route-speed controls and navigation visibility. Everything runs locally.

Start

Stop the older server with Ctrl+C, extract this full ZIP to a fresh folder and run `.\start-windows.bat` in PowerShell. Open `http://127.0.0.1:8774/`. The header should say **Terrain Studio V5**. The included prebuilt app needs Python 3; npm is only needed to rebuild source changes. Use Ctrl+F5 if an old page remains cached.

Visible panels

The camera and the labelled tab grid stay above a separate scrolling control area. The tabs wrap into three columns instead of squeezing into one row. Buttons have keyboard focus indicators; Left/Right/Home/End navigate tabs. **Hide feed** collapses the camera to make more room on short screens. All three environments use this layout. The **Tests** tab is available in Obstacle Track.

Potholes

1. Choose **Obstacle Track → Tests → Use Test Course** for the mixed course, or **Use isolated lane** for a flat paved laboratory with a separate pothole and bump section. Use the isolated lane when you want to avoid approach ramps and other course features influencing measurements.

2. Select P1 or P2. Load a shallow, broken-asphalt or deep preset, or edit radius/depth in centimetres, aspect ratio, edge transition, broken-edge amount, distance along the track and side offset.

3. Click **Apply pothole**. The room floor is cut out and a real cavity mesh is rebuilt. There is no stencil/depth-buffer trick. The pavement, cavity and tyre support share the same analytic depression. Broken lips, embedded aggregate and surface cracks provide shape cues.

4. Use **Focus pothole** to move the orbit camera to it. Enable/disable the selected pothole with its checkbox. Settings persist separately per track/seed.

5. Choose a speed under Settings, then **Run pothole approach** for a repeatable short open route. It resets the robot pose and speed without rewinding the simulation clock. It preserves recorded buffers; terrain edits and trial resets are locked while recording.

Tyre support uses the upper envelope of a rolling circle sampled across the tyre width, so a narrow depression can be partly bridged. Body height, pitch and roll remain quasistatic; a passive rim-loss approximation dissipates speed while climbing out. This is not a spring/damper suspension or deformable-soil solver. A deep hole can stall the reduced-order rover.

The second default pothole is moved slightly beyond the bump section to make the short bump approach easier to isolate. Very large or overlapping openings are rejected when applying a pothole; keep the opening within the track width and away from start/finish. Features may still be near other obstacles, so inspect placement before a trial.

Speed bumps

Select a bump section, then choose **smooth cosine hump**, **rounded rubber bump** or **flat-top speed table**. Set height, travel width, centre spacing and count. Click **Apply speed bumps**, then **Focus bumps** or **Run bump approach**.

The visible yellow/black surface uses the same height profile as the drive surface. The old intersecting cylinders are removed. Spacing must exceed width by at least 4 cm, and edits cannot overlap the next height feature or finish area. These are geometric editor limits, not road-design standards.

Route speed

Settings now has Precision (0.12 m/s), Normal (0.28 m/s), Brisk (0.40 m/s), a custom target, acceleration and braking reference limits, and two strategies:

- **Surface aware** reduces the reference on potholes, bumps, gravel, low-grip ground and narrow passages.

- **Constant reference** keeps the same requested speed across surfaces for comparisons. The existing route controller may still slow for turns and obstacles.

The reference is capped at 85% of the calibrated no-load wheel speed; it cannot guarantee that speed under load. The panel distinguishes actual speed, ramped reference and motor/profile cap. Hazard stopping can immediately zero the reference. These settings affect automatic route driving; manual arrow-key driving keeps its existing controls. Speed settings persist per environment and lock while recording.

Export speed report saves reference settings, time-weighted speed/error statistics, peak speed/planar acceleration and stop counts. These are display-step diagnostics, not a replacement for fixed-step dataset logs. Reset measurements before comparing runs.

Trial reports and recordings

Trials report duration, travelled distance, maximum speed, pitch, roll and absolute slip, with 10 Hz samples from the 120 Hz simulation hook. Export the JSON report from Tests. The isolated lane starts with a gentle 2 cm pothole and broad edges; increase severity after a baseline run. A trial can complete, be stopped, detect a persistent stall, time out, or be invalidated by a configuration/track change. It is not a physical pass/fail certification.

Trials do not automatically record camera images: use the existing Record button if you want RGB/CSV. Dataset metadata includes `terrain_tests` and `speed_control`; track ZIPs also include `track/terrain-tests.json`. Existing replay, sensor inputs/truth separation, quality checks and checksums remain.

More useful contacts for imported props

Visuals now exposes friction, restitution and contact shape:

- **Box**: cheapest conservative bounding box.

- **One convex hull**: convex approximation of all visible mesh vertices.

- **Convex parts**: up to 48 connected triangle-island hulls, preserving gaps between disconnected parts such as furniture legs. Included original furniture uses this by default.

Mass is distributed across collider volumes and total mass is retained. Complex meshes exceeding the hull budget fall back to a box; the editor reports the actual mode and part count after applying. Convex parts are not automatic concave decomposition: a connected hollow object may still have its opening filled. Instanced imported geometry is not decomposed.

Added Rapier props in Obstacle Track use a 4 cm sampled heightfield of the actual track, including potholes and bumps. The rover still uses the shared reduced-order motor/tyre model and quasistatic support; its planar proxy couples to added props. Legacy room contacts, walkers, gates and existing AABB proxies retain their limitations. ToF intersects registered visible meshes.

What still needs your real-world data

Measured room geometry, actual missing CAD parts, calibrated friction/COM, camera calibration photographs and hardware sensor logs were not supplied. They cannot be recovered accurately from this app alone. V5 does not add a room scan, automatic checkerboard solver, photoreal animated people, depth/instance-mask dataset export or full articulated vehicle dynamics. The existing tools for manual calibration and real/sim comparison remain available.

See `VALIDATION_V5.md` for the checks and remaining GPU verification limit. Model/material source credits remain in `public/visual/ASSET_CREDITS.md`.

8. The furniture and asset library

V6 adds 20 original room models to the existing eight, for **28 selectable furniture/prop models plus the rover**. All V5 terrain tests, speed controls, visible navigation, V4 PBR assets and V3 research tools are retained. The complete modelling source, 29 standalone GLBs, geometry previews and prebuilt app are included.

New models

Category	Models
Bedroom	Full-length standing mirror; double bed with duvet and pillows; two-drawer bedside cabinet; wardrobe
Office	Writing desk with drawer/cable tray; bookcase with books and decor
Living room	Lounge armchair; six-seat dining table; media console with television
Kitchen	Upholstered counter stool; refrigerator; oven/hob; sink cabinet and worktop
Appliances	Front-load washing machine
Lighting	Floor lamp with fabric shade; ceramic table lamp
Bathroom	Vanity with basin; close-coupled toilet
Fixtures	Panel radiator with fins/valves; coat rack with hanging jacket

These are original parametric models, with rounded forms, hardware, trim and material roles. Dimensions and masses are editable starting assumptions. Verify measurements for your actual objects before treating a scene as hardware evidence. Bulb meshes, screens and appliance controls are visual details; the models do not add functioning appliances or new room lights.

Choose and place

1. Open a room, then **Visuals**. The model library appears near the top.

2. Search names such as `bed` or `mirror`, or filter by category. The picker shows the selected model's actual GLB geometry preview, nominal dimensions and mass.

3. Click **Add model near robot**. New objects are offset using their footprint to avoid initially overlapping the rover where space permits.

4. Select **Place in 3D**. The app opens Orbit view; click the floor to position the selected object. Esc cancels. **Snap placement** rounds X/Z to a 10 cm grid. Track placement uses the local terrain height.

5. Use **Rotate -15° / +15°**, or enter exact X, vertical height, Z and yaw in the object editor. Set width/height/depth in metres, then **Apply object**. The fields specify the object's overall bounds.

6. **Duplicate object** creates an independent copy that you can place separately. **Remove object** deletes only the selected scene addition. Use the scene-object menu to switch between added objects.

All placements, dimensions, rotations, mass, material/contact settings and mirror preference persist locally per room. Export a **scene ZIP** for a portable backup. Existing scene ZIPs remain compatible;

scenes using V6 model IDs require V6 or later. The scene limit remains 40 added props, so the expanded catalogue does not load all 28 meshes at once.

Mirror

The standing mirror includes a frame, back panel, hinges, stand and mirror surface. By default it uses the bundled environment reflection. Select it and enable **Live room reflection** for a planar reflection of the room:

- Only one mirror is live at a time; enabling another disables the previous one.

- Balanced uses a 512 px render target, High 1024 px. Low disables live reflections while retaining the preference.

- A live mirror adds rendering cost. Leave it off when you need higher capture throughput.

- The standalone GLB carries the reflective material and surface metadata; the simulator supplies the optional live-reflection behaviour.

- ToF still intersects geometric surfaces. Reflected sensor-light paths, glass transmission and real mirror range failures are not modelled.

Generated reflection render targets are omitted from scene archives; the base model and preference are retained. GPU reflection appearance still needs checking on the laptop.

Source and offline operation

`src/room-models.js` contains the 20 new models; `src/visual-models.js` retains the original eight and rover. `tools/build-visual-models.mjs` rebuilds the GLBs. The optional CPU preview tool uses NumPy/Pillow/trimesh and renders actual triangle depth, with no network requests. These build dependencies are not needed to run the prebuilt app.

The app and all visual assets run locally. Python 3 serves `dist/` through `.\start-windows.bat` . Source edits need `npm ci` and `npm run build`. Stop the old server, extract this ZIP into a fresh folder and look for **Asset Studio V6** in the header. Default local login remains `admin / robot123`.

Original model source and GLBs use the project MIT license. Bundled texture/HDRI credits remain in `public/visual/ASSET_CREDITS.md`. Read `VALIDATION_V6.md` for checks and limits.

9. Detailed people and scene arrangement

Run the updated download

1. Extract the entire ZIP into a **new folder**, for example `Downloads\robolab-v7`.

2. Open that folder in PowerShell. Run the launcher using this PowerShell syntax:

```
.\start-windows.bat
```

3. Open the local URL printed in the terminal. Default: `http://127.0.0.1:8774/`.

4. Default local UI login: `admin` / `robot123`. Choose your environment.

5. Keep the terminal open. Use Ctrl+F5 once if an old tab still shows V6.

The ZIP contains current **source code and a rebuilt** `dist`. Running the BAT uses `dist`; changing source in VS Code requires `npm ci` then `npm run dev`, or `npm run build` to update `dist`. Opening `index.html` directly is not supported. Python 3 is required for the BAT. No SAM, CUDA, Isaac, Blender or AI account is needed to run this update. Python image tools are only needed to regenerate asset previews, not to run the app.

What changed

- Primitive people are replaced by bundled, textured anatomical character meshes with skinned rigs. Sam and Maya include authored walking motion. Sam also has a seated pose. Existing room walkers and the controlled human use this system.

- Six People catalogue entries: Sam, Maya, their walking-in-place presets, seated Sam and a detailed static laboratory researcher. The original 28 furniture/prop models remain.

- The original dark studio layout remains. In **Visuals**, use **Browse realistic people** to jump to the People category.

- **Arrange your scene** edits both original furniture groups and added models with coloured Move / Rotate / Scale handles, exact numbers, snapping, undo/redo and reset.

- Room layouts auto-save per environment. Scene ZIP export/import also contains original-furniture transforms, alongside the existing added-model data.

Add and place a person

1. Open **Visuals → Browse realistic people**.

2. Choose **Sam · realistic person** or **Maya · realistic person**. Click **Add model near robot**.

3. In **Arrange your scene**, choose the added person from **Editable scene object**.

4. Click **Enable scene editing**. The robot is paused. Use Move handles: red X, green Y (height), blue Z. Drag empty space to orbit.

5. Use **Rotate** for orientation and **Scale** for size. Hold on to a handle and drag; these are real scene transforms, not changes to a flat image.

6. **Place on floor** aligns the model's lowest bounding point to the local terrain. **Finish scene editing** hides the handles. Press **Pause** to resume the robot when ready.

The regular **Object editor** remains available for mass, material, contact shape, duplicate, remove, and dynamic prop settings. Select the added human there before using the **Human pose** card. Walk in place animates the figure without inventing a path. To make an existing pedestrian travel, use **Build → People → Set position / Walk to / Set route**.

Rearrange original furniture

Choose an entry beginning **Room ·** in Arrange your scene, or enable editing and click its visible geometry. Move sofas, desks, tables, chairs and other furniture groups. The canteen's table-with-chairs groups move together as originally constructed. This editor preserves that grouping; it does not split every mesh into independent furniture pieces.

Exact position, angles and scale are inside an expandable section to keep the panel compact. X and Z are horizontal scene coordinates; Y is up. Rotation numbers are degrees. Scale factors are positive multipliers. Non-uniform scaling changes proportions: use equal X/Y/Z factors to preserve a person's anatomy. Exact dimensions for added models are still in Object editor.

Snapping uses 10 cm translation, 15-degree rotation and 0.1 scale increments. Turn snapping off for fine placement. Ctrl+Z undoes a transform; Ctrl+Shift+Z redoes it. T, R and S select Move, Rotate and Scale while editing. Esc during a drag cancels it; otherwise Esc exits editing. Reset transform returns to the pose first selected in the current session. It is not a destructive factory reset of all saved data.

Walls, scripted gates/doors, moving carts and pedestrians are deliberately excluded from this furniture list because other code owns their movement. People already have position and route controls in Build. Added people are ordinary selectable scene models. Imported GLBs remain editable through the same object controls. This is scene arrangement, not polygon sculpting, automatic house reconstruction or a full skeleton-pose editor.

Saving and restoring a layout

The added-model store remains IndexedDB, scoped to the current browser origin and environment. Original-room transforms use a separate per-environment localStorage key. A changed original piece is held at its chosen height to avoid legacy gravity immediately undoing placement. **Export scene ZIP** is the portable backup; it includes a layout array in `visual/scene.json`. Import it into the same environment to restore those transforms and add its models.

Scene imports are additive for added models, as in V6. Importing the same ZIP twice creates two copies of added models. Original furniture transforms are applied to their stable room IDs. Older V6 scene ZIPs without a layout array continue to import. The 40-added-object and embedded-GLB import limits remain. A different browser or port has different local storage, so keep a scene ZIP backup.

How the human code works

`boot.js` loads two local GLBs before creating a room. `realistic-human.js` holds reusable templates. `SkeletonUtils.clone` gives each character an independent skeleton; its immutable geometry and textures are shared. Materials are cloned so changing one person's shirt does not recolour another person. `human.js` retains its original synchronous API: `g`, `meshes`, `pose`, `sit`, `mats`. Existing callers can therefore register actual mesh triangles for sensing and animate people without replacing their routes.

An AnimationMixer samples the original walking clip. The old controller advances gait phase in proportion to travel speed; the new sampling converts that phase using the source clip's stated 1.2 m/s nominal travel equivalent. This is a practical animation synchronization rule, not calibrated foot-contact dynamics. Stopping samples a standing pose. Sam's matching Sit_Stand clip was merged into his GLB

without copying a second complete texture set. Maya has no seated clip in this package. The researcher is a static, textured figure.

These are authored anatomical models built from MakeHuman/MPFB CC0 assets, with original styling and authored motion by Innerscene. They are **not scans of real people**, medical anatomy models or motion-capture validation. Skin, face, hair and clothing have much more detail than the old primitive figures. The app does not add soft-tissue physics, cloth simulation, finger control, facial expressions, ragdolls or human balance dynamics. Dynamic Rapier props still use approximate rigid colliders; making a person a dynamic prop does not create a biomechanical human.

How editing works

`scene-transform.js` validates finite position, unit quaternion and positive scale, and stores transform history. `scene-editor.js` owns selection, raycasting, TransformControls handles, keyboard shortcuts and compact number fields. Its event handler is registered before legacy room-drag handlers, so an enabled edit gesture is not also interpreted as camera orbit or old furniture dragging. Editor visuals use the main-view layer and are omitted from robot-camera images. Generated gizmos are marked editorAuxiliary for scene archival.

`visual-studio.js` combines added-object records with a list of original furniture. Added-model commits rebuild their Rapier contact shape and persist their object record. Original-room commits save a layout and refresh environment proxies. While the direct editor is enabled, its added-prop physics steps and old move gizmo are suspended. Recording guards prevent geometry changes during a recording. Close editing and resume the robot before collecting a new run.

Source map

File	Purpose
src/realistic-human.js	Offline loading, independent rigs, walking/standing/seated samples, People catalogue
src/human.js	Original API preserved; fallback geometry if asset loading fails
src/boot.js	Preloads local characters before room construction; visible fallback warning
src/scene-transform.js	Finite transform validation and bounded undo/redo history
src/scene-editor.js	Direct selection, handles, exact fields, floor placement, shortcuts
src/visual-studio.js	Furniture/people catalogue, room-layout persistence, existing prop physics and scene ZIP
src/main.js, canteen.js, track.js	Supply editable furniture groups; retain room-specific behaviour
tools/build-human-assets.py	Reproducible source downloads, embedded 1K textures, merged Sam sit animation
tools/human-gltf-tools.mjs	Node geometry/rig inspection without DOM image decoding
tools/bake-human-previews.mjs	Samples actual geometry for CPU preview generation
tools/render-human-previews.py	Static UV-textured CPU previews, independent of browser graphics
tests/people-editor.test.mjs	Offline rig, animation independence, transform round-trip and history checks
public/visual/humans/	Bundled GLBs, source URLs, license credits and SHA-256 hashes

Performance and privacy

Embedded textures are limited to 1024 pixels on the longest edge. The complete visual asset collection is about 48.4 MiB uncompressed, including the original furniture/materials; its V7 regression ceiling is 64 MiB. These human meshes are more detailed than the original figures, so adding many people increases draw and skinning work. Start with a few people and Balanced graphics; lower graphics settings and capture resolution if recording misses deadlines. A universal FPS guarantee is not possible. Existing sensor/physics simplifications remain unchanged.

All character loading uses local paths inside the extracted download. Runtime does not send your recordings or scenes to an asset service or AI endpoint. Build-time asset regeneration downloads public assets; the source URLs in the credits are documentation. Keep the default loopback server binding for laptop-only use. The local UI login is a convenience screen, not a network security boundary.

10. Worked learning exercises

Exercise A: calculate expected wheel speed

At radius 0.0675 m and 60 RPM, calculate peripheral no-load speed. Multiply circumference 2*pi*r by one revolution per second. Answer: approximately 0.4241 m/s. Now double the robot mass while retaining its motors: no-load wheel speed is unchanged, but acceleration and load behaviour can change. This separates kinematics from dynamics.

Exercise B: find a synchronization error

A ToF observation is acquired at 2.000 s and delivered at 2.035 s. A camera frame is captured at 2.020 s. Its latest delivered ToF packet cannot be this future-delivered observation. Align by what was available at capture, preserve acquisition/delivery stamps and keep sensor age. Never simply choose the closest timestamp without deciding whether future information is allowed.

Exercise C: identify leakage

Your target is forward velocity, but the input includes the simulator's exact velocity or a caption saying the robot's speed. The test score can look excellent without solving perception. Choose observations appropriate to the task, keep truth/teacher labels separate, and split at an independent environment/run level.

Exercise D: move a model properly

Move a chair visually and leave its physics collider at the old location. What happens? The robot can hit empty space or pass through the visible chair. The V7 commit path rebuilds added-object contact state and refreshes original-world proxies. Changing only mesh.position is not a complete editor integration.

Exercise E: repeat a pothole experiment

Use the same route, robot calibration, initial state and nominal speed. Record three baseline trials. Increase only pothole depth. Compare duration, pitch, roll, slip and stall detection. Report the configured shape, support approximation and sample timing. Do not infer real hardware safety from the result.

Exercise F: explain a human animation limitation

A rigged character looks natural while walking but still slides sideways when dragged or intersects a chair. The animation controls appearance; the scene transform and approximate colliders control placement/contact. The project does not solve human balance or continuous cloth collision. Better appearance does not establish human biomechanics.

A practical study schedule

Session 1: run the app and locate every panel. Session 2: follow boot and room initialization. Session 3: reproduce speed/radius calculations and inspect dynamics. Session 4: collect a short dataset and explain every file. Session 5: calibrate from a controlled measurement. Session 6: rearrange a room and repeat a held-out experiment. Keep a notebook of parameters, assumptions and observed failures.

Appendix A. Actual source excerpts

These excerpts come from the updated source. Original line numbers appear at the left. Long lines wrap for print; copy executable code from the ZIP rather than from a wrapped PDF listing.

A1. Pose and impulse helpers

integratePose applies a constant-twist arc update. impact1D validates masses/restitution and applies equal/opposite impulse. The distance and angle arguments already include the time integration; they are not speeds.

src/dynamics.js (original lines 25-34)

```
25    const scale=Math.abs(angle)<1e-8?1:Math.sin(angle/2)/(angle/2), mid=pose.yaw+angle/2;
26    pose.x+=distance*scale*Math.cos(mid);pose.y+=distance*scale*Math.sin(mid);pose.yaw+=angle;
27    return pose;
28  }
29  export function impact1D(m1,m2,u1,u2,e) {
30    if(!(m1>0&&m2>0&&e>=0&&e<=1))throw Error('Invalid impact parameters');
31    const J=(1+e)*(u1-u2)/(1/m1+1/m2);
32    return {v1:u1-J/m1,v2:u2+J/m2,impulse:J};
33  }
34  export function fallingStep(y,v,h,{g=GRAVITY,restitution=0,floor=0}={}) {
```

A2. Fixed-step advance

Allowed elapsed time enters an accumulator. Each loop spends one STEP, updates physics and calls the supplied hook. The clock increments after that hook, which is why recording integration must handle callback timing carefully.

src/dynamics.js (original lines 48-59)

```
48    // Sign is electrical command; speed error changes motor no-load RPM, not the ground pose.
49    advance(dt, commands, {pitch=0,roll=0,surface=null,error=[0,0],brake=false,hold=false,onStep=null}={})
    {
50      if(!Number.isFinite(dt)||dt<0)throw Error('dt must be finite and nonnegative');
51      const allowed=Math.min(dt,.1);this.droppedTime+=dt-allowed;this.accumulator+=allowed;
52      while(this.accumulator+1e-12>=STEP){
53        this.lastOptions={pitch,roll,surface:surface?{...surface}:null,error:[...error],brake,hold};
54        this.substep(STEP,commands,this.lastOptions);if(onStep)onStep(STEP,this);
55        this.accumulator=Math.max(0,this.accumulator-STEP);this.time+=STEP;this.steps++;
56      }
57      return this;
58    }
59    substep(h,commands,{pitch,roll,surface,error,brake,hold}) {
```

A3. Validated transforms

Transforms preserve orientation as a quaternion, not only yaw. This shared contract rejects non-finite values, invalid rotation and non-positive scale before applying state. The history class later in the same source uses these snapshots for Undo and Redo.

src/scene-transform.js (original lines 1-11)

```
1  // Small renderer-independent contract shared by scene UI, history and tests.
2  export function readTransform(object){return
   {position:object.position.toArray(),quaternion:object.quaternion.toArray(),scale:object.scale.toArray()}}
3  export function validateTransform(t){
4    for(const [key,length]of [['position',3],['quaternion',4],['scale',3]])
5      if(!Array.isArray(t?.[key])||t[key].length!==length||t[key].some(v=>!Number.isFinite(v)))throw
   Error('Transform must contain finite position, rotation and scale');
```

```
  6   if(Math.abs(t.position[0])>50||Math.abs(t.position[2])>50||t.position[1]<-2||t.position[1]>10)throw
    Error('Keep objects within 50 m horizontally and -2 to 10 m in height');
  7   if(t.scale.some(s=>s<.01||s>100))throw Error('Scale must be positive and between 0.01 and 100');
  8   if(Math.abs(t.quaternion.reduce((s,v)=>s+v*v,0)-1)>.01)throw Error('Rotation must be a unit
    quaternion');
  9   return t;
 10 }
 11 export function writeTransform(object,t){validateTransform(t);object.position.fromArray(t.position);objec
    t.quaternion.fromArray(t.quaternion);object.scale.fromArray(t.scale);object.updateMatrixWorld(true)}
```

A4. Independent human instances

A shared template is cloned with SkeletonUtils so bones belong to the new character. Each instance gets independent material objects; immutable geometry and texture data can remain shared.

src/realistic-human.js (original lines 26-45)

```
 26 // immutable textures are shared, keeping several walkers practical on a laptop.
 27 export function createRealisticHuman(options = {}) {
 28   const id = options.character || (options.female ? 'maya' : 'sam');
 29   const template = templates.get(id);
 30   if (!template) return null;
 31   const model = SkeletonUtils.clone(template.scene);
 32   const g = new THREE.Group(), facing = new THREE.Group();
 33   g.userData.human = true;
 34   g.userData.humanSource = 'CC0 MakeHuman / MPFB character by Innerscene';
 35   facing.rotation.y = Math.PI/2; // source +Z forward -> rover-world +X forward
 36   facing.add(model); g.add(facing);
 37   const meshes = [], materials = new Map();
 38   let shirt = null, pants = null;
 39   model.traverse(o => {
 40     if (!o.isMesh) return;
 41     meshes.push(o);
 42     const own = m => {
 43       if (!materials.has(m)) materials.set(m,m.clone());
 44       const clone = materials.get(m);
 45       clone.userData.surface = /body|eyebrow|teeth|lash/.test(m.name) ? 'skin' : 'fabric';
```

A5. Camera projection contract

The intrinsics calculation converts FOV to pixel focal length. D and K are exported with explicit optical axes and mount coordinates. The distortion helper works on normalized image coordinates.

src/camera-model.js (original lines 3-3)

```
  3 export function intrinsics(p,width,height){const fy=height/2/Math.tan(p.fov*Math.PI/360);return{width,hei
    ght,fx:fy,fy,cx:p.cx*width,cy:p.cy*height,distortion_model:'plumb_bob',D:[p.k1,p.k2,p.p1,p.p2,0],K:[fy,0,
    p.cx*width,0,fy,p.cy*height,0,0,1],frame_id:'camera_optical',optical_axes:'x right, y down, z
    forward',mount_body_xyz_m:[p.x,p.y,p.z],mount_yaw_pitch_roll_deg:[p.yaw,p.pitch,p.roll]};}
```

A6. CSV escaping

The encoder preserves an empty value for null/undefined, quotes fields containing commas/quotes/newlines and doubles embedded quotation marks. This prevents captions from breaking the CSV column structure.

src/dataset-export.js (original lines 1-3)

```
  1 import JSZip from 'jszip';
  2 import {validateDataset,sha256,parseCSV,splitFor,VERSION,BUILD_ID} from './lab-core.js';
  3 export function csvText(columns,rows){const quote=value=>{const s=String(value??'');return
    /[",\r\n]/.test(s)?'"'+s.replaceAll('"','""')+'"':s;};return
    '\ufeff'+[columns,...rows].map(row=>row.map(quote).join(',')).join('\n');}
```

Appendix B. File and concept atlas

Source module	Purpose
src/app-random.js	Named seeded random streams
src/asset-core.js	Catalogue filtering and safe placement helpers
src/boot.js	Login, environment selection and human preload
src/cad-assets.js	Original STL/URDF inspection and replacements
src/camera-model.js	Camera calibration and image effects
src/canteen.js	Canteen scene and recording loop
src/collide.js	Legacy rover/object momentum response
src/collider-parts.js	Bounded connected-island convex contact shapes
src/curtains.js	Window curtain interaction
src/dataset-export.js	CSV partitions, ZIP files, quality and hashes
src/directions.js	Navigation helpers
src/dynamics.js	Fixed-step motor, tyre, body and battery state
src/human.js	Human factory API and fallback geometry
src/lab-core.js	Sensors, parsing, calibration, comparison, quality, splits and replay
src/lab.js	Research UI and recording integration
src/lighting.js	Room lighting controls
src/main.js	Apartment scene and recording loop
src/mirror-view.js	One optional live planar mirror and resource cleanup
src/move3d.js	Legacy object movement and falling
src/narrate.js	Local explanatory narration
src/paint.js	Material colour controls
src/people.js	Walking routes, positions, modes and pedestrian controls
src/physics.js	Shared gravity and prop estimates
src/prop-physics.js	Rapier world, colliders and robot coupling
src/realistic-human.js	Detailed rigs, animation sampling and People catalogue
src/research-tools.js	Standalone research tools

Source module	Purpose
src/research.js	Rover integration, calibration and route teacher
src/robot-model.js	Rover geometry, mounts and camera
src/roll.js	Legacy rolling-object dynamics
src/room-models.js	Twenty additional original furniture models
src/rotate.js	Legacy rotation controls
src/scene-editor.js	Selection and interactive transform handles
src/scene-transform.js	Transform validation and history
src/showroom.js	Model/material preview and download
src/speed-control.js	Target speed, strategy and rate limits
src/studio-layout.js	Fixed camera/navigation and scrolling panels
src/task.js	Tasks and route planning
src/track-geometry.js	Analytic hole/bump surface and wheel support
src/track-surface.js	Track surface integration
src/track-tests.js	Terrain trial capture and metrics
src/track.js	Obstacle-track scene and recording loop
src/verification.js	Numerical verification page
src/visual-core.js	Geometry normalization and GLB validation
src/visual-models.js	Original detailed furniture and rover meshes
src/visual-studio.js	PBR materials, catalogue, scene imports and prop records
src/weights.js	Original mass controls
src/wheelerr.js	Motor mismatch controls

Glossary

Term	Easy meaning
State	The current numbers describing a simulated thing
Mesh	Vertices and triangles describing a visible surface
Material / PBR	Surface appearance parameters used by lighting
Rig / skin weights	Bones and per-vertex influences that deform a character
Collider	Simplified shape used for contact calculations
Pose / transform	Position, orientation and scale of an object
Odometry	Estimated motion integrated from wheel observations
Ground truth	Internal simulated state used for diagnosis or labels
Latency / jitter	Delivery delay / variation in that delay
Bias / noise	Systematic offset / changing error
Quaternion	Four-number representation of 3D rotation
Fixed timestep	Equal duration used for each dynamics update
Seed	Initial value for a reproducible random sequence
Manifest	File list with sizes and content hashes
Leakage	Information in training inputs/test construction that unfairly reveals the answer
Calibration	Parameter estimation from measured evidence
Validation	Checking whether behaviour matches the intended real task

Appendix C. Verification and provenance

Completed checks

- `npm test`: 82 JavaScript checks passed, including the existing drive, sensor, camera, terrain, dataset, assets and Rapier checks.

- New checks inspect the actual bundled human GLBs, verify SHA-256 values and embedded 1K texture metadata, load geometry/rigs through Three's GLTFLoader, sample authored walking and seated clips, verify finite skinned vertices, and check independent bones/materials between instances.

- Transform tests cover full quaternion/non-uniform-scale round-trip, invalid finite/bounds/scale rejection, undo/redo, failed-history retention and clearing a branched redo chain.

- Python asset/URDF suite: 8 checks passed. Missing source CAD remains explicitly disclosed.

- The production build succeeds. The ZIP contains that rebuilt dist as well as all source and required offline assets.

- CPU previews were rasterized from the actual geometry, UVs and embedded texture images. Sam, Maya and researcher previews were inspected. They are not generated concept pictures or screenshots of a WebGL renderer.

- The showroom catalogue and static human fallback preview were inspected in the available browser.

Verification limits

The available preview browser cannot create a WebGL context. Its HTTP preview hostname also does not provide WebCrypto for the simulator's secure-context login. Interactive GPU rendering, drag handles in a running room, Windows execution and performance on the user's RTX 4060 were therefore not verified here. The source preserves the normal localhost login requirement; no production authentication workaround was added for this preview environment.

No new real hardware logs were supplied. These changes improve character geometry and scene arrangement, not physical validation. The new figures use authored animation and approximate rigid contact shapes; they do not add cloth dynamics, ragdolls or a biomechanical human solver.

Local acceptance check

1. Run the complete fresh ZIP using `.\start-windows.bat`. Confirm the V7 header.

2. Open Visuals, browse People, add Sam and Maya, and verify their textures.

3. Add a walking preset and observe limb animation when simulation steps run.

4. Enable scene editing, select an existing chair, drag one axis, rotate and scale it. Check Undo and Redo. Test Esc during a drag.

5. Select an added person, set numeric X/Y/Z and equal scale factors, and place it on the floor. Finish editing, then resume the robot.

6. Reload the room and check saved placement. Export a scene ZIP and check that visual/scene.json contains layout and objects.

7. Record a short camera/telemetry run and inspect images and quality.json before starting a large data collection.

Attachment provenance

Input: robolab-sim(1).zip. SHA-256: e278065d012bd3b32bdf22cf5b9beb62ba3ba66686eeb9e8fe169178afa0189d. The V7 deliverable is a separate updated archive; this manual does not alter the uploaded original.

Human asset provenance

sam - https://www.innerscene.com/tools/library/3d-parts/walk-carrying-phone-sam-3d-person-animated-241a2abe

Bundled SHA-256: b0da9b3ef1add2c579786bdb860e303ec7374bebdb22aaf93f550b8ac2c11eff. CC0-1.0; authored anatomical character, not a real-person scan.

maya - https://www.innerscene.com/tools/library/3d-parts/walk-carrying-phone-maya-3d-person-animated-06ba936a

Bundled SHA-256: dfca43ce48a72f764f72dea446f87db2c5ce849ca78144271db0ed92bb45524d. CC0-1.0; authored anatomical character, not a real-person scan.

researcher - https://www.innerscene.com/tools/library/3d-parts/laboratory-researcher-3d-person-healthcare-7a3ceb81

Bundled SHA-256: 4535082d32c633f7fd9c95b7e38409dbdc349f0bad00104a7aa883d76a8e220d. CC0-1.0; authored anatomical character, not a real-person scan.

Source references

The primary implementation references are the files in the source atlas, the original URDF/STL assets and the versioned project guides reproduced and explained here. The following official references are additional learning material; APIs can differ from the older bundled Three r128 version.

- Three.js materials: https://threejs.org/docs/pages/MeshStandardMaterial.html

- Three.js glTF loading: https://threejs.org/docs/pages/GLTFLoader

- Rapier rigid bodies: https://www.rapier.rs/docs/user_guides/javascript/rigid_bodies/

- Rapier colliders: https://www.rapier.rs/docs/user_guides/javascript/colliders/

- ROS URDF physical properties: https://docs.ros.org/en/rolling/Tutorials/Intermediate/URDF/Adding-Physical-and-Collision-Properties-to-a-URDF-Model.html

- CC0 human asset dedication: https://creativecommons.org/publicdomain/zero/1.0/

Retain the complete source ZIP, package-lock.json, measured calibration, scene ZIP and raw recordings together. A document explains the implementation; the executable source and recorded evidence remain the reference for reproducing an experiment.